CYBER PANCHAYAT · TEAM HANDBOOK

Git Workflow Guide

Task Division & Commit Messages · Clone → Branch → Commit → Push → Pull Request → Merge

0. Task Division — Who Owns What

Each person works in their own branch (see table below) so changes rarely touch the same files. Check off each task as you complete and test it. Suggested commit messages are given for every task — use them as-is or adapt the wording, but keep the same one-commit-per-task idea.

Pammi — Complaints & Assignments

Branch: `feature/complaints`

Citizen complaint lifecycle · Staff assignment handling

TASKS

- ☐ Submit Complaint form (category, AI suggest, paper scan)
- ☐ My Complaints list (filter, delete own pending complaint)
- ☐ Complaint Detail page (timeline, resolve/reopen/close)
- ☐ Public Track Complaint page
- ☐ Staff dashboard + My Assignments + Update Status
- ☐ Admin: assign/reassign officer, reject complaint, budget tag

FILES

`NewRequest.vue` `MyRequests.vue` `ComplaintDetail.vue` `TrackComplaint.vue` `staff/*.vue` `admin/ManageRequests.vue`

Backend: `Request*Resource` `Assignment*Resource` `AISuggestResource`

SUGGESTED COMMIT MESSAGES

```
git commit -m "Add submit complaint form with category selection"
git commit -m "Add AI category suggestion and paper-scan OCR flow"
git commit -m "Add My Complaints list with filter and delete-pending"
git commit -m "Add complaint detail page with status timeline"
git commit -m "Add resolve/reopen/close actions to complaint detail"
git commit -m "Add public track-complaint page"
git commit -m "Add staff dashboard and My Assignments view"
git commit -m "Add admin assign/reassign officer, reject and budget-tag actions"
```

Rithvik — Facilities, Bookings & Payments

Branch: `feature/facilities-bookings`

Facility booking flow · Payment collection

TASKS

- ☐ Browse facilities + availability calendar

FILES

`FacilitiesView.vue` `MyBookings.vue` `MyPayments.vue`

- ☐ Create booking (date/time/attendee validation, conflict check)
- ☐ My Bookings (cancel pending, view invoice)
- ☐ Payments page (pay pending, view history)
- ☐ Admin: Manage Facilities (CRUD), Manage Bookings (confirm/cancel/complete)

FILES

`admin/ManageFacilities.vue`
`admin/ManageBookings.vue`
Backend: `Facility*Resource`
`Booking*Resource` `Payment*Resource`
`BookingInvoiceResource`

SUGGESTED COMMIT MESSAGES

```
git commit -m "Add facilities list with availability calendar"
git commit -m "Add booking form with date/time and attendee validation"
git commit -m "Add booking conflict check for overlapping time slots"
git commit -m "Add My Bookings page with cancel and invoice view"
git commit -m "Add payments page with pay-now and history"
git commit -m "Add admin Manage Facilities CRUD"
git commit -m "Add admin Manage Bookings confirm/cancel/complete actions"
```

Nilesh — Community, Announcements & Notifications

Branch: `feature/community`

Social feed · Official announcements · Notification center

TASKS

- ☐ Community feed (create post, like, comment, delete own)
- ☐ Announcements page (admin publishes, everyone reads)
- ☐ Notifications panel + mark-as-read
- ☐ Staff community posts + Admin community moderation

FILES

`CommunityFeed.vue`
`AnnouncementsView.vue`
`NotificationsView.vue`
`staff/StaffCommunityPosts.vue`
`admin/CommunityManagement.vue`
Backend: `Post*Resource`
`PostComment*Resource`
`PostLikeResource`
`Notification*Resource`

SUGGESTED COMMIT MESSAGES

```
git commit -m "Add community feed with create post and like"
git commit -m "Add comment thread with delete-own-comment"
git commit -m "Add announcements page for admin publishing"
git commit -m "Add notifications panel with mark-as-read"
git commit -m "Add staff community posts view"
git commit -m "Add admin community moderation controls"
```

Bipin — Admin Panel, Reports & Auth

Branch: `feature/admin-panel`

Departments/officers/users management · Analytics · Login/Register

TASKS

FILES

- ☐ Manage Departments (CRUD, duplicate-name guard)
- ☐ Manage Officers (create staff account, assign department)
- ☐ Manage Users (role/ward edit, enable/disable)
- ☐ Reports & Analytics charts + Admin Dashboard stats
- ☐ Login / Register / Forgot Password / Landing page

`admin/ManageDepartments.vue`
`admin/ManageOfficers.vue`
`admin/ManageUsers.vue`
`admin/ReportsAnalytics.vue`
`admin/AdminDashboard.vue`
`LoginView.vue` `RegisterView.vue`
Backend: `Department*Resource`
`Officer*Resource` `AdminUser*Resource`
`ReportsResource`
`Register/LoginResource`

SUGGESTED COMMIT MESSAGES

```
git commit -m "Add Manage Departments CRUD with duplicate-name guard"
git commit -m "Add Manage Officers with department assignment"
git commit -m "Add Manage Users with role/ward edit and disable"
git commit -m "Add reports and analytics charts"
git commit -m "Add admin dashboard stats"
git commit -m "Add login, register and forgot-password pages"
```

Sakshi — Premium Subscription

Monthly billing · Paywall · Admin visibility

Branch: `feature/subscription` (already built)

TASKS

- ☐ Subscribe / Cancel / Resume flows
- ☐ Monthly invoice generation + Pay Now + invoice view
- ☐ Paywall gating on complaints & community feed
- ☐ Admin: Manage Subscriptions page + dashboard stat + user badge
- ☐ Review, test, and extend this feature further

FILES

`MySubscription.vue`
`admin/ManageSubscriptions.vue`
Backend: `Subscription*Resource`

SUGGESTED COMMIT MESSAGES

```
git commit -m "Add subscribe, cancel and resume subscription flow"
git commit -m "Add monthly invoice generation and pay-now"
git commit -m "Add paywall gating on complaints and community feed"
git commit -m "Add admin Manage Subscriptions page and dashboard stat"
```

Shared files — coordinate before editing: `frontend/src/api/index.js` and `frontend/src/api/seed.js` (the offline mock) are touched by every feature. Add your section without rearranging existing code, and pull `main` often to catch overlaps early.

1. One-Time Setup — Clone the Repository

Each team member runs this once, on their own machine:

```
git clone https://github.com/PammiTiwari/final.git
cd final
```

> This downloads the full project with its complete history. You now have your own local copy that talks to the shared copy on GitHub (called `origin`).

2. Create Your Own Branch

Never work directly on `main`. Before starting your part of the project, create a branch named after your feature:

```
git checkout main
git pull origin main   // make sure you start from the latest main
git checkout -b feature/your-feature-name
```

PERSON	SUGGESTED BRANCH NAME
Pammi	`feature/complaints`
Rithvik	`feature/facilities-bookings`
Nilesh	`feature/community`
Bipin	`feature/admin-panel`
Sakshi	`feature/subscription`

3. Make Changes, Then Commit

1. **Check what changed:** `git status`
2. **Stage your files:** `git add path/to/file1 path/to/file2` (avoid `git add .` — it can accidentally include files you didn't mean to touch)
3. **Commit with a clear message:** `git commit -m "Add facility booking calendar view"`

> **Commit often, in small chunks.** Five small commits with clear messages are far easier to review and merge than one giant commit called "update". Use the imperative mood — `"Add X"`, `"Fix Y"`, `"Update Z"` — not `"added"` or `"fixes"`. Every person's suggested commit messages are listed with their tasks in Section 0.

4. Push Your Branch

```
git push -u origin feature/your-feature-name
```

> This uploads *your branch* to GitHub — it does **not** touch `main`. The `-u` flag only needs to be used the first time; after that, plain `git push` works.

5. Open a Pull Request (PR)

On GitHub, open the repository and click **"Compare & pull request"** (it appears automatically after a push). Fill in:

- **Title** — what the change does, e.g. "Add facility availability calendar"
- **Description** — a short summary of what you built and how to test it

Then click **Create pull request**. A teammate reviews it before it merges into `main`.

6. Keep Your Branch Updated (Pull)

While you work, teammates will merge their own PRs into `main`. Regularly bring those changes into your branch so conflicts stay small and easy to resolve:

```
git checkout feature/your-feature-name
git pull origin main
```

If Git reports a conflict, it will mark the conflicting lines directly in the file:

```
<<<<<<< HEAD
your version of the code
=======
their version of the code
>>>>>>> main
```

Edit the file to keep the correct combination, remove the `<<<<<<<` / `=======` / `>>>>>>>` markers, then:

```
git add path/to/conflicted-file
git commit
git push
```

7. Merging the Pull Request

Once a PR is approved on GitHub, click **Merge pull request**. Then, everyone else should sync their own `main`:

```
git checkout main
git pull origin main
```

Command Cheat-Sheet

COMMAND	WHAT IT DOES
`git status`	Shows what's changed and what's staged
`git pull origin main`	Downloads and merges the latest main into your current branch
`git push`	Uploads your committed changes to GitHub
`git checkout -b name`	Creates and switches to a new branch
`git log --oneline -10`	Shows the last 10 commits, one line each
`git diff`	Shows exactly what lines changed, before staging
`git branch -a`	Lists all branches (local and remote)

Golden Rules

- Never commit directly to `main` — always work in your own branch.
- `git pull origin main` before you start work each day, so you catch conflicts early.
- One PR = one feature. Don't bundle unrelated changes together.
- Write commits in the imperative mood ("Add", "Fix", "Update") and one task = one commit — see Section 0 for exact suggested messages per person.
- If you're touching `frontend/src/api/index.js` or `seed.js` (shared mock files), add your section without rearranging what's already there — this is where most conflicts happen.